

Assignment - 6 (Three address code)

```
#include <iostream>
#include <stack>
#include <vector>
using namespace std;
bool isOperator(char c) {
    return (c == '+' || c == '-' || c == '*' || c == '/' || c == '^');
}
```

```
int precedence(char op) {
    if (op == '+' || op == '-') return 1;
    if (op == '*' || op == '/') return 2;
    if (op == '^') return 3;
    return 0;
}
```

```
bool isValid(string expr) {
    int balance = 0;
    for (char ch : expr) {
        if (ch == '(') balance++;
        else if (ch == ')') balance--;
        if (balance < 0) return false;
    }
    return balance == 0;
}
```

```
string infixToPostfix(string expr) {
    stack<char> st;
    string postfix = "";

    for (char ch : expr) {
        if (isdigit(ch)) {
            postfix += ch;
        }
        else if (ch == '(') {
            st.push(ch);
        }
        else if (ch == ')') {
            while (!st.empty() && st.top() != '(') {
                postfix += st.top();
                st.pop();
            }
            st.pop();
        }
        else {

```

```

        while (!st.empty() && precedence(st.top()) >= precedence(ch)) {
            postfix += st.top();
            st.pop();
        }
        st.push(ch);
    }
}

while (!st.empty()) {
    postfix += st.top();
    st.pop();
}

return postfix;
}

void generateTAC(string postfix) {
    stack<string> st;
    int tempCount = 1;

    cout << "\nThree Address Code (TAC):\n";
    cout << "-----\n";

    for (char ch : postfix) {
        if (isalnum(ch)) {
            st.push(string(1, ch));
        }
        else {
            string op2 = st.top(); st.pop();
            string op1 = st.top(); st.pop();

            string temp = "t" + to_string(tempCount);

            cout << temp << " = " << op1 << " " << ch << " " << op2 << endl;

            st.push(temp);
            tempCount++;
        }
    }
}

int main() {
    string expression;

    cout << "Enter an arithmetic expression: ";
    cin >> expression;

    if (!isValid(expression)) {

```

```
        cout << "Invalid Expression!" << endl;
        return 0;
    }

    string postfix = infixToPostfix(expression);

    cout << "\nPostfix Expression: " << postfix << endl;

    generateTAC(postfix);

    return 0;
}
```

Input:

A+B*C

Output:

Postfix Expression: ABC*+

Three Address Code (TAC):

t1 = B * C

t2 = A + t1